Lab 3 Report: Wall-Following Robot

Team 13 - TAMJAM

Tissany Chen
Jeryl Lewis
Ansis Anderson
Muktha Ramesh

Robotics: Science and Systems

March 14, 2026

1 Introduction

In Lab 3, our team worked on implementing our wall-following code from Lab 2 onto the physical robot and designing a safety controller. Wall-following is a form of autonomous robot navigation that involves using sensors and algorithms to detect walls and a controller to maintain a consistent distance away while driving alongside the wall. The robot uses LIDAR scan data to filter for relevant points and detect the positions of walls relative to itself, from which the drive controller issues a new drive commands.

The wall follower must be configurable, capable of navigating both straight lines and corners, and recover from unfavorable states quickly and reliably. To achieve this, the wall follower has to be tuned to the physical robot setup.

Additionally, a safety controller based on the LIDAR scan data was implemented to protect the robot from any crashes during the development and deployment of the autonomous driving code. We developed two versions of the safety controller, based on rectangular and Ackermann driving safety zones, respectively.

The following report will discuss the solution approach and evaluation of our wall followers and safety controllers.

2 Safety Controller Approach

2.1 Problem Statement

Purpose
The safety controller stops the robot whenever the LiDAR data signal that the current drive command will result in contact with an object. This ideally works both for static objects (walls, chairs, etc.) and dynamic objects (a person walking in front of the robot), and even if a crash is unavoidable, it slows the robot down enough to prevent damage.

Design Objectives

- **Reliability** - The safety controller should be robust enough to the point where the operator feels comfortable not stopping the robot manually in crash situations.
- **Precision** - The safety controller should still allow the robot to take tight corners by walls and not restrict available maneuvers.
- **Computational Speed** - the latency from running the safety controller should not negatively impact the performance of the robot.

2.2 Solution Approach/Implementation

2.2.1 A Kinematics Based Safety Controller

The safety controller sends a safety command if the LiDAR scan detects any objects within a certain threshold distance of the robot's current trajectory. By assuming constant deceleration, the safety controller uses kinematic equations to determine threshold distance d_t, given the robot's current velocity v, max deceleration a_{max}, and a constant safety margin m that accounts for noise, controller latency, and modeling imperfections.

$$d_t = m + \frac{v^2}{2a_{max}} \tag{1}$$

If any LiDAR data is within the threshold distance d_t, a safety command that sets the robot's speed to 0 and keeps its commanded steering angle δ is sent.

Algorithm 1 Safety command sent to the robot by the safety controller

Require: steering angle δ
1: **function** SAFETYCOMMAND(δ)
2: drive_speed $\leftarrow 0.0$
3: drive_angle $\leftarrow \delta$
4: **end function**

We arrived at two main implementations to decide if any LiDAR is within d_t. The more straight-forward implementation checks for points inside a rectangle in

front of the car, while the implementation more accurate to the car's trajectory checks for points in a region that curves based on the turning angle of the car.

2.2.2 Rectangular Safety Zone

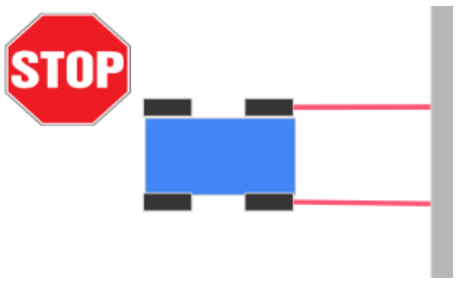

Figure 1: Activating the safety command using the rectangular safety zone

In this approach the LiDAR scan points are converted into cartesian coordinates centered on the front bumper. We draw our safety zone as a rectangle with the same width as the car and with a length of d_t. If any points are detected inside this rectangle, the safety controller sends the safety command.

Algorithm 2 Detecting points in rectangular safety zone

Require: car width W, steering angle δ, threshold distance d_t, LiDAR scan P

function EVALUATESAFETYRECTANGLE(W, δ, d_t, P)
  $P_{cart} \leftarrow$ TOCARTESIAN(P)
  if $(p \in P_{cart} \mid |p_y| \leq W/2 \ \ \& \ \ 0 \leq p_x \leq d_t)$ **then**
    SAFETYCOMMAND(δ)
  end if
end function

2.2.3 Ackermann Steering Safety Zone

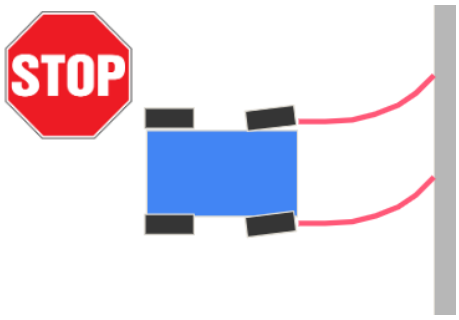

Figure 2: Activating the safety command with the ackermann steering safety zone

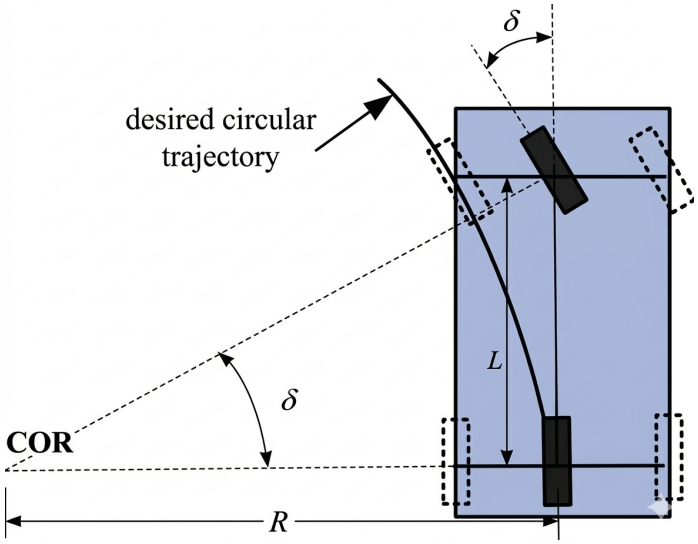

Figure 3: Ackermann steering geometry from COR perspective. (Adapted from: X. Li, Z. Sun, Q. Chen, and D. Liu, "An adaptive preview path tracker for off-road autonomous driving," 2013 10th IEEE International Conference on Control and Automation (ICCA), pp. 1718–1723, Jun. 2013)

In this approach, the LiDAR scan points are converted into COR (center of rotation) polar coordinates. We can use the simple bicycle model with the car length L to calculate the radius of the car's center in this frame:

$$R = \frac{L}{\tan(\delta)} \tag{2}$$

The safety zone is constructed as the swept arc between the outer circle that intersects the front outer tire (with radius R_{max}), and the inner circle that intersects the back inner tire (with radius R_{min}).

$$R_{min} = R - \frac{W}{2} \tag{3}$$

$$R_{max} = \sqrt{L^2 + (R + \frac{W}{2})^2} \tag{4}$$

The distance of each point is calculated as the arc distance between the angle of the front bumper and the angle of the point. If any point has a radius between R_{min} and R_{max} and a distance within d_t, the safety command is sent.

Algorithm 3 Detecting points in ackermann steering safety zone

Require: car width W, steering angle δ, threshold distance d_t, LiDAR scan P

1: **function** EVALUATESAFETYACKERMANN(W, δ, d_t, P)
2: $P_{COR} \leftarrow$ TOCOR(P)
3: **if** $(p \in P_{COR} \mid R_{min} \leq p \leq R_{max}$ & $0 \leq$ ARCDISTANCE(p) $\leq d_t)$ **then**
4: SAFETYCOMMAND(δ)
5: **end if**
6: **end function**

Note that for small δ these two approaches are actually equivalent. Moving forward we used the ackermann steering safety zone because it met our objectives of being more precise with an insignificant increase in computational cost.

2.3 Experimental Evaluation

2.3.1 Straight Trajectory Test

We started testing our safety controller by sending our car into a wall straight in front of it at different velocities, tracking its distance from the wall over time (closest 5% of points in front of the car to account for noise).

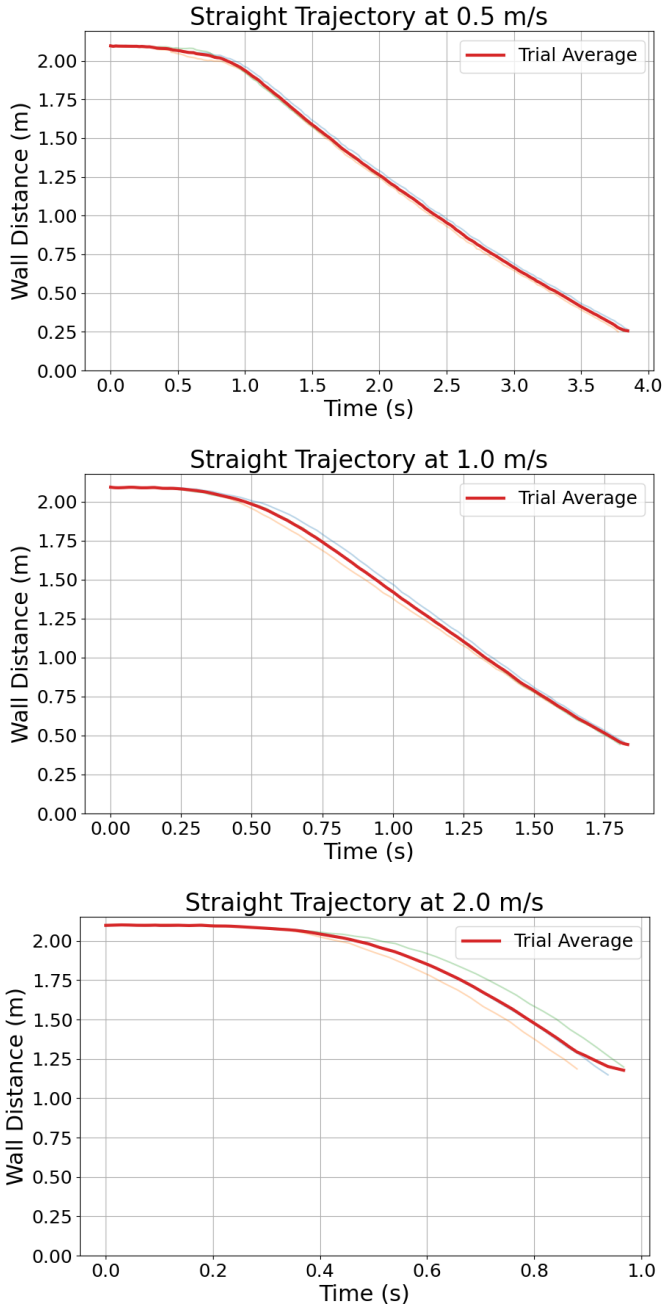

Figure 4: Straight trajectory tests show that a_{max} varies with velocity. Tested with $\delta = 0$, $a_{max} = 2.0$, $m = 0.2$ for 3 trials each

While our controller worked for various velocities, a_{max} increased with velocity (The distance we stopped at became increasingly larger than our margin). We may need to calibrate a_{max} based on our speed to get the right sensitivity.

2.3.2 Curved Trajectory Test

We also demonstrated the need for the ackermann steering safety zone by conducting a similar experiment with $\delta = 0.3$, a near max turn. The safety controller would trigger with the rectangle safety zone, but our chosen method allows for the tight turn, stopping the car before running into another wall.

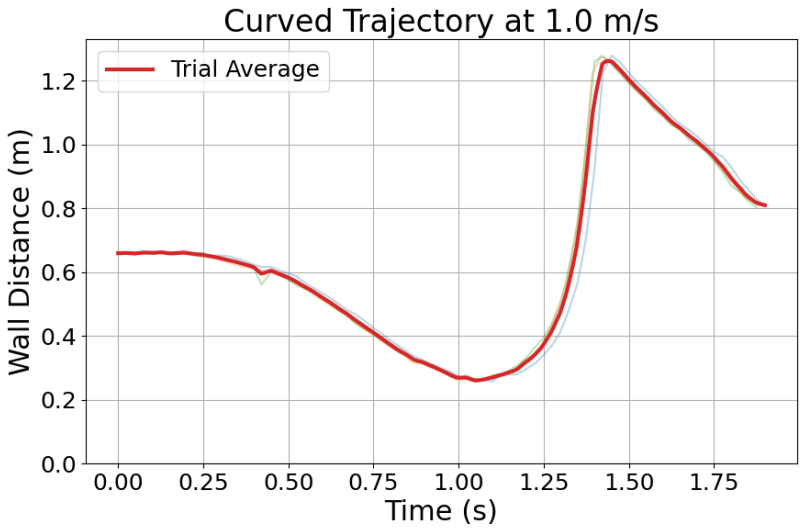

Figure 5: Curved trajectory test shows real world benefit of ackermann steering's increased precision. Tested with $\delta = 0.3$, $a_{max} = 2.0$, $m = 0.2$ for 3 trials

3 Wall Follower Approach

3.1 Problem Statement

The objective of the wall-following system is to autonomously drive a robot parallel to a wall while maintaining a fixed target distance from it. The robot must continuously estimate the position of the wall using LIDAR data and compute steering commands that minimize deviation from the desired distance. Let d denote the measured perpendicular distance from the robot to the wall and $d_{desired}$ the target distance. The control objective is to minimize the error

$$e(t) = d_{desired} - d(t) \tag{5}$$

while maintaining stable motion and avoiding oscillations.

The controller must operate in real time and remain robust when encountering environmental variations such as corners, sensor noise, or partial occlusions of the wall.

We evaluated these goals both qualitatively and quantitatively. Qualitatively, we plotted the distance from the wall over time and visually inspected whether the robot followed lines marked on the ground. Quantitatively, we measured the consistency of the wall distance using metrics like average distance error, mean absolute error (MAE), and the standard deviation of the distance.

3.2 System Overview and Initial Setup

The wall-following system runs on a differential-drive robot equipped with a 2D LiDAR sensor. The LiDAR produces a sequence of range measurements describing the surrounding environment.
Our system processes these data in three stages:

1. Filter LiDAR scan data to isolate points corresponding to the wall
2. Estimate the wall geometry relative to the robot
3. Compute a steering command that maintains the desired wall distance

3.3 Wall Estimation from LiDAR

Accurately estimating the position of the wall is essential for reliable wall-following. Several strategies were explored to extract a wall model from the LiDAR scan data.
First, scan data were filtered to include only points corresponding to the target wall. This reduced interference from objects located on the opposite side.

Several line-estimation techniques to model the wall were then evaluated:

- Principal Component Analysis (PCA) and Iterative End Point Fit (IEPF) based line estimation
- Linear regression on filtered scan points
- Linear regression on the closest 10% of scan points

From the resulting line parameters, wall orientation and the perpendicular distance to the wall can be computed.

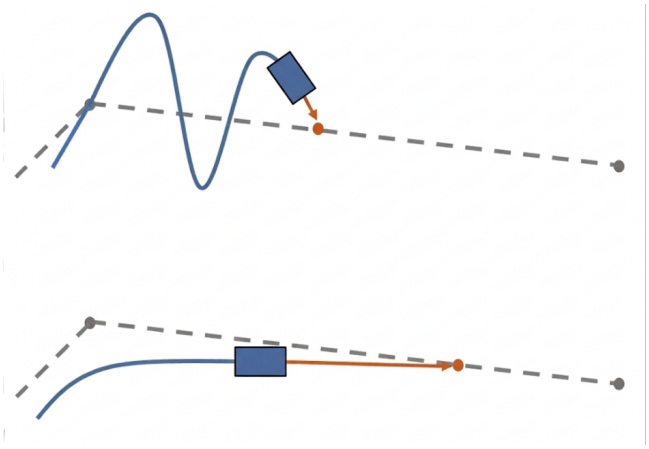

Figure 6: Visualization of pure pursuit controller: The red dot represents the target point on the path and the blue line represents the steering curvature computed to intercept that point.

3.4 Controller Design

Several control strategies were explored for maintaining the desired wall distance.

Pure Pursuit

The pure pursuit controller treats the wall as a path to be followed. A target point is selected along the wall at a fixed lookahead distance, and the robot steers toward this point.

Proportional Controllers with Derivative Terms

Two proportional variants used wall slope or angular difference as derivative terms to encourage alignment.

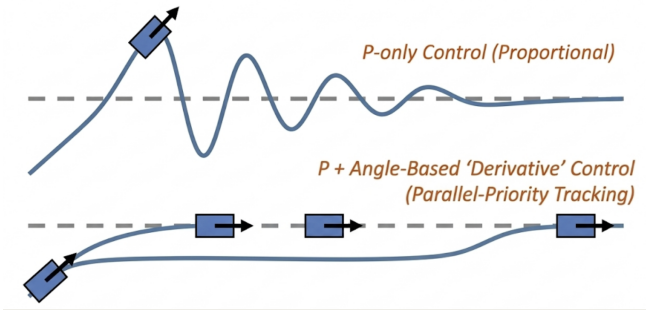

Figure 7: The top visualization shows standard proportional control. The bottom visualization shows additional derivative approximations to dampen oscillations.

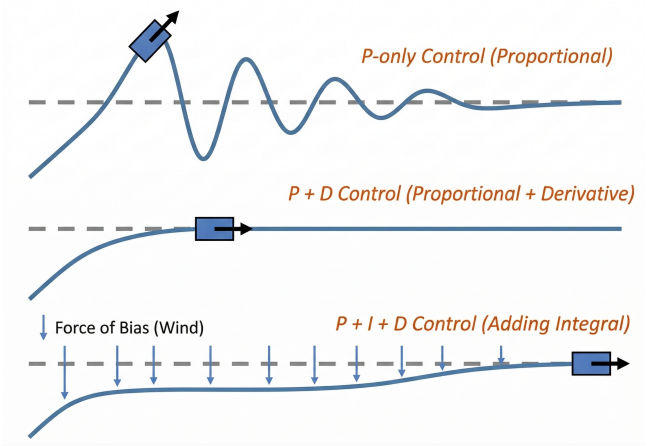

Figure 8: The top visualization shows standard proportional control, the medium visualization shows the oscialliation-dampening effect of adding derivative control, and the bottom visualization shows that integral control can be added to eliminate steady-state distance error caused by system bias.

PID Control
The final controller uses a PID formulation:

$$\delta = K_P e(t) + K_I \int e(t)\,dt + K_D \frac{de(t)}{dt} \tag{6}$$

where δ is the steering angle and K_P, K_I, and K_D are the controller gains. The proportional term corrects distance errors, the integral term removes steady-state error, and the derivative term dampens oscillations.

3.5 Real-World Approach

When transitioning to physical robot experiments, we encountered several challenges. Our codebase was not highly modular, particularly in separating wall estimation from the controller, which made merging individual implementations difficult. Additionally, each team member used a different filtering method tailored to their specific controller. As a result, we tuned each implementation individually and selected the best-performing robot based on the evaluation metrics.

Determining Ground Truth
A major challenge in real-world testing was determining a reliable ground-truth measurement for the robot's wall distance. Three approaches were considered:

1. Manually stopping the robot and measuring the distance from the wall.

2. Evaluating each wall-estimation algorithm using static tests and selecting the most accurate one as the ground truth.

3. Using the median output of all four wall-estimation algorithms.

We selected the third approach. The first option was impractical due to the large amount of manual effort required and the potential for human measurement error. The second option risked biasing the evaluation towards a single algorithm.

Instead, we recorded ROS bag files during each trial run. For every laser scan message, we computed the median distance estimate across all four wall-estimation algorithms. This approach reduced noise and prevented bias toward any single method.

3.6 Experimental Evaluation

Simulation Testing
Simulation experiments were conducted using the six test cases provided in the `wall_follower_sim` repository, with six performance metrics. Each robot implementation used its own wall-estimation method to determine the distance from the wall. Angle measurements were included because the robot's orientation relative to the wall affects the stability of wall-following behavior.

Simulation Results
The following table summarizes the averaged results across all six simulation test cases.

Controller	Avg Dist Err	MSE	Dist Std	Std Err	Avg Angle	Angle Std
PID + PCA + IEPF	0.27275	0.60526	0.56337	0.03819	0.19226	0.30557
P + Angle Deriv + LR	0.40045	1.04395	0.71236	0.03428	0.13010	0.15977
P + Slope Deriv + LR	0.39733	1.27138	0.83657	0.04539	0.12300	0.17172
Pure Pursuit + LR	0.31119	1.60796	0.70884	0.03175	0.25404	0.30924

Table 1: Simulation results averaged across six test cases. The PID controller has the lowest average distance errors, mean squared errors, and wall distance standard deviations.

Full results:[Link]

One limitation was that each team member measured wall distance differently, making it difficult to compare results. Additionally, simulation results differed from real-world performance.

Despite these limitations, the simulation experiments provided insight into which controllers might perform best in practice. The PID controller performed best on the distance-based metrics, although each controller demonstrated different strengths.

Testing Procedure
Experiments were conducted in two stages.
First, we evaluated all robots using straight-line tests. Each test case was repeated three times.

- Left wall following, inward starting angle, 0.6 m from the wall, speed 0.5 m/s
- Right wall following, outward starting angle, 0.6 m from the wall, speed 1.0 m/s

The two robots with the lowest mean absolute error (MAE) were then selected for corner testing.
Corner tests included four scenarios:

- Left wall, slow speed (0.5 m/s)
- Left wall, fast speed (1.0 m/s)
- Right wall, slow speed (0.5 m/s)
- Right wall, fast speed (1.0 m/s)

Each corner test was performed three times.

Straight-Line Results
In the straight-wall experiments, both the PID and pure pursuit controllers achieved the lowest MAE values across the left and right wall tests. These two controllers were therefore selected for further testing on the corner course.

Qualitative observations supported the quantitative results. Both controllers were able to maintain the desired distance from the wall more consistently than the other approaches.

However, the controllers that used wall angle or wall slope as derivative terms tended to drift from the desired distance during real-world testing. Encouraging the robot to remain parallel to the wall dominated the distance error, causing the robot to maintain orientation rather than distance.

For the PID controller, large oscillations were observed during the high-speed tests, and one trial resulted in the robot colliding with the wall.

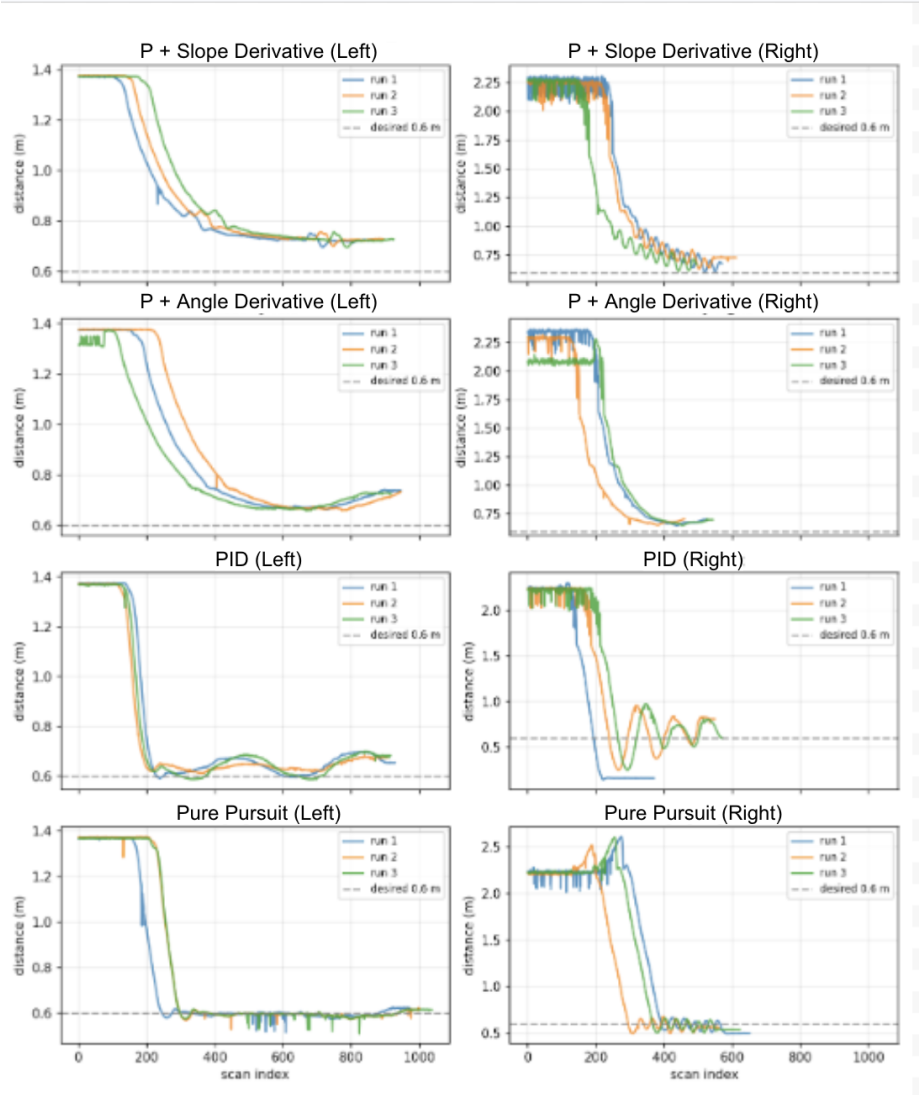

Figure 9: Straight line tests show that each controller performed differently, especially in regards to error from desired wall distance and oscillation tendencies.

Controller	Mean Dist. (m)	Std Dev (m)	Min (m)	Max (m)	MAE (m)
Left Wall Following – Slow Speed (0.5 m/s)					
PID	0.77696	0.27469	0.58675	1.37679	0.17780
Pure Pursuit	0.77888	0.31967	0.50956	1.37543	0.19017
P + Slope Derivative	0.91333	0.25100	0.69699	1.37844	0.31333
P + Angle Derivative	0.88378	0.27094	0.66025	1.37897	0.28378
Right Wall Following – Fast Speed (1.0 m/s)					
PID	1.21352	0.80091	0.14420	2.30107	0.76070
Pure Pursuit	1.41494	0.80719	0.49824	2.60784	0.84865
P + Slope Derivative	1.40416	0.68682	0.60618	2.31383	0.80416
P + Angle Derivative	1.36237	0.69873	0.64841	2.36855	0.76237

Table 2: Quantitative performance metrics for the straight-line wall-following experiments. Each controller was evaluated over three trials, and distance measurements were aggregated using the median of four wall-estimation algorithms to reduce noise. The table reports the mean distance from the wall, standard deviation, minimum and maximum observed distances, and mean absolute error (MAE) relative to the desired wall distance. Lower MAE values indicate more accurate wall tracking, while lower standard deviation indicates more stable behavior with fewer oscillations. In the slow-speed left-wall test, the PID and pure pursuit controllers achieved the lowest error values, while the derivative-based proportional controllers exhibited larger deviations from the desired distance. At higher speeds during right-wall following, all controllers experienced increased variability, though the PID controller maintained comparatively strong distance-tracking performance.

The pure pursuit controller exhibited smooth behavior during slow tests and small but frequent oscillations during faster tests. In the straight-wall scenario, pure pursuit best satisfied the design goals of maintaining distance while minimizing oscillations.

Corner Testing

During corner testing, we observed that the pure pursuit controller took excessively large turns when navigating corners. This caused the robot to deviate significantly from the desired distance.

After several trials, we determined pure pursuit was not suitable for the corner route, as it took large turns and was slow to return to the desired wall distance.

In contrast, the PID controller consistently returned to the desired distance much more quickly after encountering corners. Although oscillations were still visible during faster runs, the controller remained stable and avoided collisions.

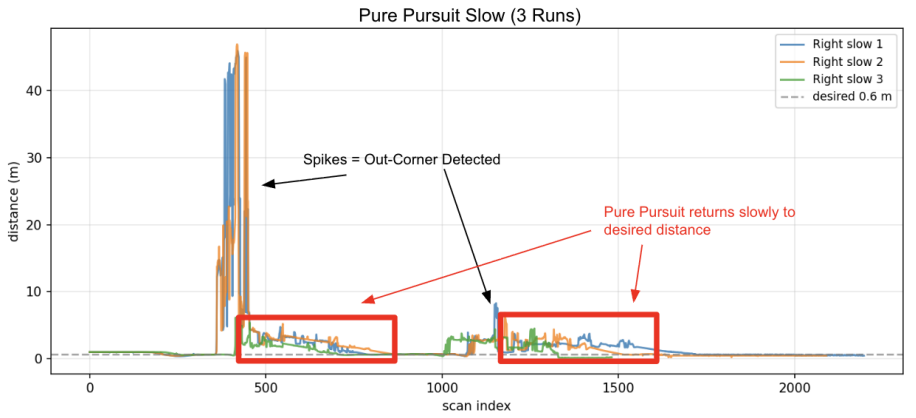

Figure 10: Pure Pursuit Graph - This is three runs of the pure pursuit controller in the long route test case. The spikes correspond to when the robot notices that there is an out-corner. The red boxes indicate the time it takes for the robot to get back to the desired distance after encountering an out-corner.

Final Decision
The PID controller was ultimately selected as the final wall-following algorithm. It was the only controller that consistently maintained the desired distance and successfully navigated both straight and corner environments.

Although the PID controller exhibited oscillations at higher speeds, we believe these could be reduced through further tuning of the PID gains.

4 Conclusion

In this lab, we've successfully implemented both the wall follower and the safety controller, and evaluated each to ensure their performance. Our safety controller, designed with both rectangular and Ackermann Steering safety zones, met our design objectives of reliability, precision, and speed. We determined that the PID wall-follower that used a PCA-based filtering and IEPF line estimation performed well on both desired distance metrics and corner tests, despite larger oscillation amplitudes that can be tuned in the future. The safety controller is crucial to autonomous robot navigation and the wall follower is a possible implementation of it. In the next lab, we will consider how camera input can be processed to further aid autonomous driving.

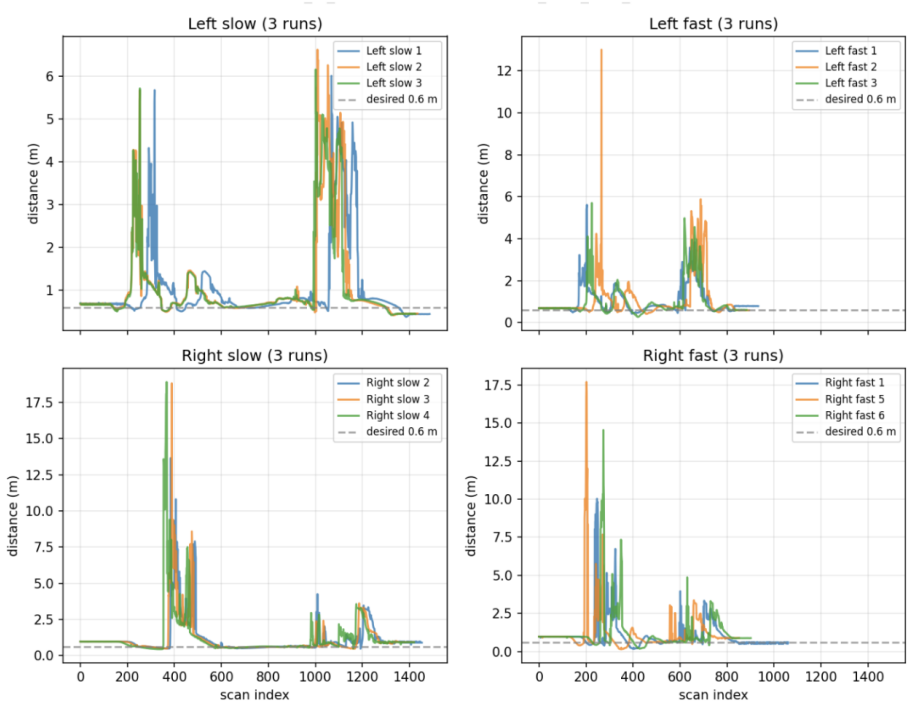

Figure 11: PID Graphs on Long Route - In particular, comparing spikes in the the trial results for the "right wall, slow" test case with the Pure Pursuit controller reveals a quicker return to desired distance.

5 Lessons Learned

5.1 Tissany's lessons learned

I learned that debugging work on the robot involves understanding how real life impacts what the program is trying to do. For example, I learned that gains often need to be lowered considerably when transitioning from simulation to real life, likely because of delays or other noisy factors that the controller shouldn't respond too aggressively to. Another example is looking from the point of view of the robot to debug. It took us a while to catch that there were persistent anomalies in the visualization that messed with the transition from simulation to real life, and that this was due to a wire blocking the LIDAR's field of view, because we weren't paying attention to possible hardware bugs.

On the communication side, it was revealing to see that there were certain pieces of information that we took for granted, because we were working within our own perspective and did not consider the audience's point of view. For example, it did not occur to me that the audience would be curious about the names that we labeled our graphs with and had to mentally match our names with the wall follower implementation.

5.2 Jeryl's lessons learned

Through this lab I learned how difficult the transition from simulation to hardware can be. A lot of the assumptions that worked in simulation failed when it came to the real world, and I didn't anticipate how much time it would take to properly tune things to the hardware. I also learned the importance of collecting and evaluating lots of data early. I think that some of the insight that we gained from data we analyzed could have been used to further improve our design if we had more time or looked at the data earlier on in the process. It also would have given us more time to improve the visuals for our briefing and report.

On the CI side of things I learned how important it is to divide work between teammates and parallelize tasks. The timeline for meeting deadlines in this class is very unforgiving and I think that over the course of this lab we realized how important it is for us to properly distribute work so everything can get done. I sometimes had tunnel vision and spent too much time on a task that I should have moved on from. I also learned how much time goes into creating good slides. I think we pushed off slide creation too long and we weren't able to properly show the audience our work.

5.3 Ansis' lessons learned

The biggest thing I learned was just working with a physical car in general - the response times, the need for optimization, the LIDAR inconsistencies all led to lessons learned after hours of debugging. Additionally, while I technically knew

GitHub already, I would say I familiarized myself with it a lot more, since while our setup was simple in theory (one branch per person), in practice we constantly ran into weird situations or edge cases which needed specific commands to be resolved (for an example, a branch insisting its up to date with origin but still missing the latest commits). Its been a while since I have worked on a relatively large scale project like this, and it was nice finally derusting.
One more lesson was that we should always remember about future assignments and the need for data collection, since we spent hours tuning the safety controller, but we tuned it off of purely visual results and did not record any data during the process.
In terms of CI lessons, I think the biggest lesson was the importance of setting up a proper calendar, or just keeping proper records in general, since at times we would forget about meetings / assignments. Combined with our unrealistic task split (initially we planned that everyone would implement their own safety controller and then we would merge them afterwards, but that would have been a huge waste of effort), this led to a bunch of forgotten deadlines at first, and long hours grinding towards the end of the lab.

5.4 Muktha's lessons learned

I learned a lot over the course of this lab. For instance, I learned that going from simulation to real world robots is a big transition. In this case, it involved adjusting the filtering, adjusting the speed of the vehicle and turning, accounting for vehicle effects such as a wire blocking the radar, tuning the gains, changing which topics to subscribe to and publish to, and more. Simulation code does not just work right away. Learning about how LIDAR worked was also interesting, and helped me understand how autonomous vehicles in real life work as well.

On the communication side, I realized how important it is to have a schedule and try to keep to it. We needed team members to post deadlines in the group chats and we had to be very structured when coming up with when we wanted to work on which parts in order to actually get the project done on time. We didn't do this at the beginning of the lab, and I believe that's why we were a little rushed at the end. This is something that in lab 4, we have been very careful about because we have learned our lesson from lab 3.